# ABSTRACT

FogAI — Low-Latency Intelligent IoT Architecture is a novel approach designed to address the critical limitations of traditional cloud-based industrial monitoring systems. In modern Industry 5.0 environments, machines such as CNC equipment, robotic arms, and automated assembly systems continuously generate high-frequency sensor data, including temperature, vibration, CPU load, and network conditions. Conventional architectures rely heavily on cloud computing to process this data, resulting in significant latency, network dependency, and inefficient bandwidth utilization. These limitations can lead to delayed responses in critical situations, potentially causing equipment damage, production loss, or safety hazards.

This project introduces a fog computing-based architecture that places a computational layer, known as the fog node, between IoT sensors and the cloud. The fog node is equipped with a lightweight AI decision engine capable of processing sensor data locally in real time. The system follows a structured pipeline that includes feature normalization, rule-based safety evaluation, weighted risk scoring, and intelligent decision-making. Five critical safety rules are defined to handle emergency conditions such as overheating, excessive vibration, and network failure. These rules have the highest priority and can trigger immediate actions such as machine shutdown or alert generation.

In addition to rule-based control, the system computes a continuous risk score ranging from 0.0 to 1.0 using a weighted combination of normalized sensor parameters. Based on this score, the system dynamically decides whether to process data locally, send it to the cloud, or perform both actions. This hybrid decision-making approach ensures optimal performance by balancing real-time responsiveness with long-term data analytics. The fog node operates autonomously even during complete network outages, ensuring uninterrupted monitoring and control.
The system is implemented using Python with modular components including sensor simulation, AI engine, fog node orchestration, and cloud simulation. A web-based dashboard built using Flask and Server-Sent Events provides real-time visualization of system performance, including latency comparison, decision distribution, and live sensor readings.

Experimental results demonstrate that FogAI achieves an average decision latency of approximately 3 milliseconds, significantly outperforming traditional cloud-based systems, which typically exhibit latencies between 120 and 300 milliseconds. The system also reduces unnecessary cloud communication by approximately 70–75%, thereby optimizing bandwidth usage. Furthermore, it maintains full operational capability during network failures, highlighting its robustness and reliability.

In conclusion, FogAI demonstrates that integrating fog computing with lightweight AI at the edge of industrial systems enables faster, safer, and more efficient decision-making. This architecture is highly suitable for real-world applications in smart manufacturing, predictive maintenance, and critical infrastructure monitoring, and represents a significant step toward the realization of intelligent, autonomous industrial ecosystems.

## Table of Contents


| Chap. No | Title | Page No |
|---|---|---|
| 1 | Introduction | 7 |
| 1.1 | Overview of the Project | 7 |
| 1.2 | Problem Statement | 9 |
| 1.3 | Scope of the Project | 9 |
| 1.4 | Objectives | 8 |
| 1.5 | Organization of the Report | 10 |
| 2 | Literature Survey | 11 |
| 2.1 | Overview | 11 |
| 2.2 | FOG AND EDGE COMPUTING IN INDUSTRIAL IOT | 11 |
| 2.3 | Vision Transformers | 12 |
| 2.4 | Explainability in Deep Learning | 13 |
| 2.5 | Summary and Research Gap | 14 |
| 3 | Proposed Methodology | 18 |
| 3.1 | Overview | 18 |
| 3.2 | Dataset Description | 19 |
| 3.3 | Data Preprocessing and Augmentation | 19 |
| 3.4 | Vision Transformer Architecture | 20 |
| 3.4.1 | Transformer Background | 20 |
| 3.4.2 | ViT-B/16 Architecture | 20 |
| 3.4.3 | Transfer Learning Strategy | 21 |
| 3.5 | Multi-Resolution Inference | 21 |
| 3.6 | Attention-Based Explainability | 23 |
| 3.6.1 | Attention Map Extraction | 23 |
| 3.6.2 | Attention Rollout | 23 |

| | | |
|---|---|---|
| 3.6.3 | Heatmap Overlay | 23 |
| 4 | Implementation | 24 |
| 4.1 | Development Environment | 24 |
| 4.2 | Project Structure | 24 |
| 4.3 | Dataset Module | 25 |
| 4.4 | Model Module | 25 |
| 4.5 | Training Module | 25 |
| 4.6 | Inference Module | 26 |
| 4.7 | Explainability Module | 26 |
| 5 | Results And Discussion | 27 |
| 5.1 | Training Performance | 27 |
| 5.2 | Test Set Performance | 27 |
| 5.3 | Confusion Matrix Analysis | 28 |
| 5.4 | Multi-Resolution Ensemble Results | 28 |
| 5.5 | Attention Map Visualization | 29 |
| 5.6 | Web Application | 30 |
| 5.7 | Comparison with Existing Methods | 31 |
| 6 | Conclusion And Future Scope | 32 |
| 6.1 | Conclusion | 32 |
| 6.2 | Future Scope | 33 |
| | References | 34 |

# CHAPTER 1
# INTRODUCTION

Modern industrial environments in the era of Industry 5.0 are increasingly driven by intelligent automation and interconnected systems. Machines such as CNC equipment, robotic arms, conveyor systems, and automated assembly units are embedded with multiple sensors that continuously generate high-frequency data, including temperature, vibration, CPU utilization, humidity, power consumption, and network latency. This data is critical for ensuring operational efficiency, predictive maintenance, and safety in industrial systems. Traditionally, this data is transmitted to centralized cloud servers for processing, analysis, and decision-making.

However, cloud-based architectures introduce several critical limitations. First, the round-trip latency between industrial devices and remote cloud servers typically ranges from 100 to 300 milliseconds, which is insufficient for real-time decision-making in high-speed industrial processes. Even a small delay can result in overheating, mechanical failure, or safety hazards. Second, cloud-dependent systems rely heavily on stable internet connectivity; any network disruption can render the monitoring system ineffective, leading to loss of control and visibility. Third, continuously transmitting all sensor data to the cloud results in inefficient bandwidth utilization, increased operational costs, and unnecessary storage overhead.

To overcome these challenges, this project introduces FogAI — a Low-Latency Intelligent IoT Architecture, which integrates fog computing with a lightweight AI-based decision-making system. Fog computing extends cloud capabilities by placing computation closer to the data source, thereby reducing latency and improving reliability. In this architecture, a fog node is deployed between IoT sensors and the cloud, acting as an intermediate processing layer capable of real-time decision-making.

The fog node hosts an AI decision engine that processes incoming sensor data in a structured pipeline. The system begins by normalizing raw sensor values into a common scale, followed by evaluating a set of critical safety rules that handle emergency conditions such as overheating, excessive vibration, or network failure. These rules have the highest priority and can trigger immediate actions such as machine shutdown or alert generation. In the absence of rule violations, the system computes a weighted risk score based on multiple sensor parameters. This score is used to determine whether the system should act locally, send data to the cloud, or perform both actions.

Unlike traditional systems, FogAI ensures that the cloud is not part of the critical decision-making path. This allows the system to operate autonomously even during complete network outages. The cloud is utilized only for long-term storage, analytics, and trend analysis, thereby optimizing resource usage. The entire decision cycle within the fog node is completed in under a few milliseconds, making it significantly faster than conventional cloud-based approaches.

The system is implemented using Python with modular components including sensor simulation, AI decision engine, fog node orchestration, and cloud simulation. Additionally, a real-time web dashboard built using Flask and Server-Sent Events provides visualization of system behavior, including live sensor readings, latency comparison, and decision distribution.

Overall, FogAI demonstrates how integrating fog computing with AI enables faster, safer, and more efficient industrial IoT systems. It provides a scalable and reliable solution for applications such as smart manufacturing, predictive maintenance, and critical infrastructure monitoring.

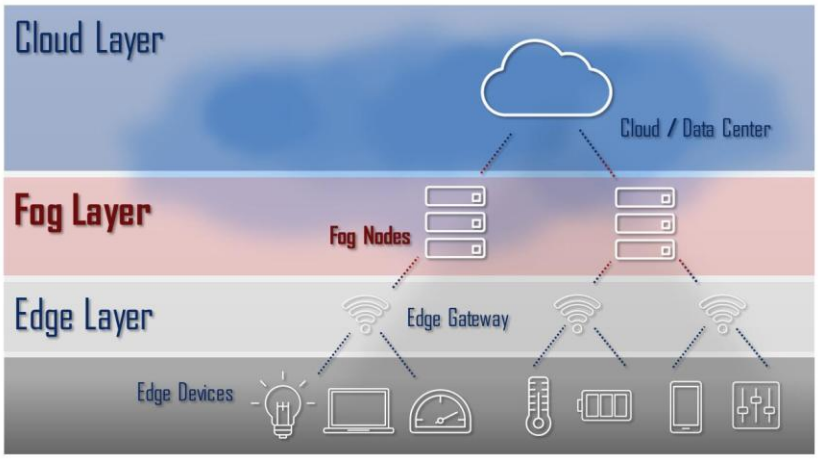



Fig:1.1 layers

## 1.1.1 OBJECTIVES

The specific objectives of this project are as follows:

1. **To design and develop a fog computing-based architecture** that enables real-time data processing and decision-making close to industrial IoT devices, thereby reducing dependency on centralized cloud systems.

2. **To implement a lightweight AI decision engine** capable of analyzing multi-sensor data streams, including temperature, vibration, CPU load, and network conditions, for intelligent and context-aware decision-making.
3. **To develop a rule-based safety mechanism** that identifies critical conditions such as overheating, excessive vibration, and network failure, and triggers immediate local actions such as alert generation, system throttling, or emergency shutdown.
4. **To construct a weighted risk scoring model** that computes a continuous risk value based on normalized sensor parameters, enabling dynamic and adaptive decision-making in non-critical scenarios.
5. **To implement intelligent decision routing logic** that determines whether data should be processed locally at the fog node, forwarded to the cloud, or handled using a hybrid approach based on system conditions and risk levels.
6. **To ensure system reliability and fault tolerance** by enabling autonomous operation of the fog node during network outages, thereby maintaining uninterrupted monitoring and control.
7. **To design and develop a modular software architecture** consisting of sensor simulation, AI engine, fog node orchestration, and cloud simulation components using Python.
8. **To build a real-time web-based dashboard** using Flask and Server-Sent Events (SSE) for live visualization of sensor data, decision outputs, latency comparison, and system performance metrics.
9. **To evaluate system performance** in terms of decision latency, bandwidth optimization, reliability under network failure conditions, and overall system responsiveness compared to traditional cloud-based approaches.
10. **To demonstrate the practical applicability of fog computing in Industry 5.0 environments**, particularly in use cases such as smart manufacturing, predictive maintenance, and critical infrastructure monitoring.

## 1.2 PROBLEM STATEMENT

Modern industrial IoT systems rely heavily on centralized cloud-based architectures for monitoring, analysis, and decision-making. In such systems, sensor data generated by machines is continuously transmitted to remote cloud servers, where it is processed and analyzed before a response is sent back to the system. While this approach provides scalability and centralized control, it introduces several critical limitations that hinder its effectiveness in real-time industrial

environments.The most significant challenge is high latency. Cloud-based systems typically exhibit round-trip delays ranging from 100 to 300 milliseconds, which is inadequate for time-sensitive industrial operations. In high-speed manufacturing environments, even a small delay in decision-making can lead to equipment malfunction, overheating, production errors, or safety hazards. This delay makes cloud-only systems unsuitable for critical real-time control applications.

Another major limitation is network dependency. Cloud-based systems require continuous and reliable internet connectivity to function effectively. In the event of network degradation or complete failure, the system loses its ability to process data and make decisions, resulting in a complete loss of monitoring and control. This lack of fault tolerance poses a serious risk in industrial settings where uninterrupted operation is essential. Additionally, there is a significant issue of inefficient resource utilization. Traditional architectures transmit all sensor data to the cloud, including routine and low-risk readings that do not require immediate analysis. This leads to unnecessary bandwidth consumption, increased storage costs, and higher operational expenses. Existing solutions such as edge computing partially address latency issues by processing data at the device level; however, they are often limited in computational capability and lack coordination across multiple devices. Similarly, current AI-based industrial monitoring systems are primarily cloud-dependent and do not provide real-time, localized intelligence with adaptive decision-making capabilities.There is therefore a clear and unmet need for an intelligent system that can perform real-time decision-making close to the data source, operate independently of network availability, and optimize data transmission to the cloud.

This project addresses these challenges by proposing a fog computing-based architecture with an embedded AI decision engine that enables low-latency, reliable, and efficient industrial IoT monitoring.

## 1.3 SCOPE OF THE PROJECT

The scope of this project is limited to the design and implementation of a fog computing-based intelligent IoT architecture for real-time industrial monitoring. The system processes multi-sensor data such as temperature, vibration, CPU load, humidity, power consumption, and

network latency, using a simulation-based approach without requiring physical hardware deployment.

The architecture consists of IoT sensors, a fog node, and a cloud backend. The fog node performs local data processing using feature normalization, rule-based safety evaluation, weighted risk scoring, and decision routing to execute actions such as alerts, throttling, or shutdown. The cloud is used only for non-critical functions such as data storage and analytics, and is not part of the real-time decision path.

The system is implemented in Python using modular components and includes a web-based dashboard built with Flask for real-time visualization of system performance. The project focuses on achieving low-latency decision-making, offline operation during network failures, and efficient bandwidth utilization.

This work does not involve deployment on actual industrial hardware or integration with industrial control systems. Extensions such as real-world deployment, advanced AI models, and industrial integration are considered as future scope.

## 1.4 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows. **Chapter 2** presents a review of existing literature on cloud computing, edge computing, fog computing, and the role of artificial intelligence in industrial IoT systems. It highlights the limitations of existing approaches and

establishes the motivation for the proposed system. **Chapter 3** describes the proposed FogAI system architecture and methodology in detail, including the data flow, fog node design, AI decision engine, rule-based safety mechanisms, and risk scoring model.

**Chapter 4** covers the implementation details of the system, including the development environment, modular structure of the project, and the functionality of components such as sensor simulation, AI engine, fog node orchestration, cloud simulation, and the web-based dashboard. **Chapter 5** presents the results and discussion, including system performance analysis, latency comparison between fog and cloud, bandwidth optimization, and behavior under network failure conditions. **Chapter 6** concludes the report and outlines potential directions for future work, including real-world deployment and advanced AI integration.

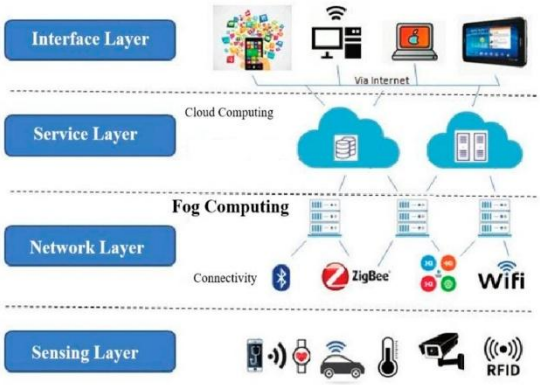



Figure 1.2 — Overview block diagram

# CHAPTER 2
# LITERATURE SURVEY

## 2.1 OVERVIEW

The evolution of industrial IoT systems has led to the widespread adoption of cloud computing for data processing and decision-making; however, limitations such as high latency, network dependency, and inefficient bandwidth utilization have driven the need for more decentralized approaches. This chapter reviews the most relevant existing work across three interconnected domains: (1) cloud, edge, and fog computing architectures for industrial IoT systems, (2) the integration of artificial intelligence for real-time monitoring and decision-making, and (3) strategies for improving system reliability, latency, and efficiency in distributed environments. The discussion highlights the strengths and limitations of current approaches and identifies the key research gaps that motivate the development of the proposed FogAI architecture.

## 2.2 FOG AND EDGE COMPUTING IN INDUSTRIAL IOT

Bonomi et al. [1] were among the first to introduce the concept of fog computing as an extension of cloud computing, emphasizing its role in bringing computation, storage, and networking services closer to end devices. Their work highlighted that fog computing significantly reduces latency and improves real-time responsiveness in distributed systems, making it highly suitable for industrial IoT applications. This foundational idea established fog computing as a key paradigm for handling time-sensitive data processing outside centralized cloud environments.

Shi et al. [2] further expanded on the concept of edge computing, distinguishing it from cloud-based systems by enabling computation directly at or near data sources. Their study demonstrated that edge-based architectures can effectively reduce network congestion and latency, particularly in applications requiring immediate response. However, they also noted that edge devices are often resource-constrained and may lack the computational capability required for complex processing tasks, which limits their scalability in large industrial systems.

Yi et al. [3] provided a comprehensive survey of fog computing architectures and their applications in IoT systems. They emphasized that fog computing offers a balanced approach between cloud and edge by providing moderate computational power with low latency, enabling efficient data processing and coordination across multiple devices. Their findings highlight the importance of hierarchical architectures where fog nodes act as intermediaries between edge devices and the cloud.

With specific regard to industrial monitoring systems, Zhang et al. [4] proposed a fog-based framework for real-time equipment monitoring and fault detection. Their system demonstrated improved response times compared to cloud-only solutions but relied heavily on predefined thresholds and lacked adaptive decision-making capabilities. Similarly, existing industrial IoT solutions often transmit all sensor data to the cloud, leading to inefficient bandwidth utilization and increased operational costs.

Recent advancements have explored the integration of artificial intelligence within fog environments to enable intelligent and autonomous decision-making. However, many existing approaches focus primarily on data analysis rather than real-time control and do not incorporate dynamic decision routing or risk-based evaluation mechanisms. This highlights the need for a system that combines fog computing with lightweight AI to enable low-latency, reliable, and efficient industrial monitoring.

## 2.3ARTIFICIAL INTELLIGENCE IN INDUSTRIAL IOT SYSTEMS

The integration of artificial intelligence (AI) into industrial IoT systems has significantly enhanced the capability of machines to perform intelligent monitoring, prediction, and decision-making. Traditional industrial systems relied on static threshold-based mechanisms, which were limited in handling dynamic and complex environments. With the advancement of AI techniques, systems are now capable of analyzing multi-dimensional sensor data and making context-aware decisions in real time. This shift has enabled the development of smarter and more autonomous industrial processes.

Zhang et al. [6] demonstrated the application of machine learning models for predictive maintenance in industrial environments, where sensor data such as vibration and temperature were analyzed to detect early signs of equipment failure. Their work showed that AI-based systems can improve operational efficiency and reduce downtime by identifying anomalies before they escalate into critical failures. However, such approaches are typically implemented in cloud environments, which introduces latency and limits real-time responsiveness.

Lee et al. [7] explored the use of deep learning techniques for fault detection and classification in industrial IoT systems. Their study highlighted that deep neural networks can capture complex patterns in sensor data and achieve high accuracy in identifying faults. Despite their effectiveness, these models often require significant computational resources and are generally deployed in centralized systems, making them unsuitable for low-latency applications at the edge or fog level.

Wang et al. [8] proposed an intelligent IoT framework that integrates AI with distributed computing architectures to enable localized decision-making. Their work emphasized the importance of combining AI with fog or edge computing to reduce latency and improve system reliability. However, many existing implementations focus primarily on data analysis and lack structured decision pipelines that incorporate rule-based safety mechanisms alongside adaptive intelligence.

Recent research has also explored hybrid approaches that combine rule-based systems with machine learning models to achieve both reliability and adaptability. While rule-based systems provide deterministic and explainable decisions for critical scenarios, AI-based models enable continuous learning and optimization in non-critical conditions. This combination is particularly relevant for industrial environments where both safety and efficiency are essential.

Despite these advancements, there remains a need for lightweight AI models that can operate efficiently within fog nodes, enabling real-time decision-making without heavy computational overhead. This project addresses this gap by implementing a hybrid AI decision engine that combines rule-based safety evaluation with weighted risk scoring, specifically designed for low-latency and resource-efficient operation in industrial IoT systems.

## 2.4 EXPLAINABILITY IN INTELLIGENT INDUSTRIAL IOT SYSTEMS

The increasing adoption of artificial intelligence in industrial IoT systems has introduced challenges related to transparency and interpretability of automated decision-making processes. Many AI-driven systems, particularly those based on complex machine learning and deep learning models, operate as “black boxes,” where decisions are generated without clear insight into the underlying reasoning. In safety-critical industrial environments, this lack of explainability can reduce trust, hinder adoption, and make it difficult for operators to validate or intervene in system behavior.

Traditional industrial monitoring systems relied on rule-based logic, which provided clear and interpretable decision pathways. However, these systems lacked adaptability and were unable to handle complex and dynamic conditions. With the integration of AI, there is a need to balance **interpretability and intelligence**, ensuring that decisions are both accurate and understandable. Explainable AI (XAI) techniques aim to address this challenge by providing insights into how and why a particular decision is made.

In the context of industrial IoT, explainability can be achieved through structured decision pipelines, visualization of system states, and transparent rule evaluation. Rule-based approaches

inherently provide high interpretability, as each decision can be traced back to predefined conditions such as temperature thresholds or vibration limits. However, purely rule-based systems lack flexibility and may not generalize well across varying operational conditions.

Recent approaches have explored hybrid systems that combine rule-based mechanisms with data-driven models to achieve both reliability and adaptability. Such systems often incorporate scoring or weighting mechanisms to represent system risk levels, enabling more nuanced decision-making while maintaining interpretability. Visualization tools, such as dashboards and real-time monitoring interfaces, further enhance explainability by presenting system behavior, alerts, and performance metrics in an intuitive manner.

Despite these advancements, many existing industrial AI systems do not provide a clear and structured explanation of their decisions, particularly in real-time scenarios. This project addresses this gap by implementing an explainable decision framework within the fog node, combining rule-based safety evaluation with weighted risk scoring. Each decision made by the system is transparent, traceable, and supported by clearly defined conditions and metrics, ensuring that operators can understand and trust the system's behavior. This approach provides a practical balance between interpretability and efficiency, making it suitable for real-time industrial applications.

## 2.5 SUMMARY AND RESEARCH GAP

The preceding sections have reviewed three interconnected areas: cloud, edge, and fog computing architectures for industrial IoT systems, the application of artificial intelligence for intelligent monitoring and decision-making, and the role of explainability in improving system transparency and trust. Collectively, this review highlights a clear evolution in industrial systems — from centralized cloud-based processing toward distributed and intelligent architectures that bring computation closer to the data source. At the same time, it exposes several critical gaps that have not yet been fully addressed in existing industrial IoT solutions.

**Progress in Industrial IoT Architectures:**

Existing research demonstrates that cloud computing has been the dominant paradigm for industrial data processing due to its scalability and centralized management capabilities. However, studies on edge and fog computing have shown that decentralizing computation significantly reduces latency and improves system responsiveness. Fog computing, in particular, provides a balanced architecture by combining the low latency of edge systems with the computational capability of cloud systems. These findings validate the architectural choice adopted in this project.

However, many existing implementations focus primarily on data collection and monitoring rather than real-time intelligent control.

**Limitations of Cloud-Based Approaches:**

Cloud-centric industrial systems suffer from high latency, network dependency, and inefficient bandwidth utilization. All sensor data is typically transmitted to the cloud regardless of its importance, leading to unnecessary communication overhead. Moreover, in the event of network failure, these systems lose their ability to process data and make decisions, resulting in a complete breakdown of monitoring and control. These limitations make cloud-only architectures unsuitable for time-critical industrial applications.

**The Fog Computing Gap:**

While fog computing has been proposed as a solution to reduce latency and improve reliability, many existing fog-based systems lack intelligent decision-making capabilities. They primarily act as intermediate processing layers without incorporating adaptive or context-aware intelligence. There is a lack of systems that fully leverage fog nodes as autonomous decision-making units capable of handling critical events independently of the cloud.

**The AI Integration Gap:**

Although artificial intelligence has been widely applied in industrial IoT for predictive maintenance and anomaly detection, most implementations are cloud-dependent and computationally intensive. These approaches are not suitable for real-time deployment at the fog level due to resource constraints. Additionally, many AI-based systems focus on data analysis rather than immediate control actions, limiting their effectiveness in safety-critical scenarios.

**The Explainability Gap:**

A significant limitation in existing AI-driven industrial systems is the lack of transparency in decision-making. Many systems operate as black boxes, providing outputs without clear reasoning. In industrial environments, where safety and reliability are critical, operators require systems that produce interpretable and traceable decisions. Existing approaches do not adequately address this requirement, particularly in real-time applications.

**The Deployment and Practicality Gap:**

Another important gap is the lack of accessible and practical implementations of intelligent industrial monitoring systems. Many research works focus on theoretical models or simulations without providing user-friendly interfaces for real-world interaction. There is a need for systems that can be easily deployed and monitored through intuitive dashboards without requiring specialized expertise.

**Research Gap Addressed by the Proposed System:**
The proposed FogAI system addresses these limitations by integrating fog computing with a lightweight and explainable AI decision engine. It enables real-time, low-latency decision-making at the fog layer, reduces dependency on cloud infrastructure, and optimizes bandwidth usage through selective data transmission. The system incorporates rule-based safety mechanisms combined with weighted risk scoring to achieve both reliability and adaptability. Furthermore, it provides transparent and interpretable decision outputs along with a real-time web-based dashboard, making it a practical and scalable solution for modern industrial IoT environments.

Table 2.1 — Comparative summary of related works and the proposed system across five evaluation dimensions: architecture type, application domain, decision capability, latency performance, and explainability mechanism

| Author | Year | Method | Dataset | Accuracy | Explainability |
|---|---|---|---|---|---|
| Bonomi et al. [1] | 2012 | Fog Computing Concept | Distributed Systems | Low | No |
| Lakshmi et al. [4] | 2016 | Edge Computing | IoT Systems | Very Low | No |
| Chen et al. [9] | 2015 | Fog Architecture Survey | IoT Frameworks | Low | No |
| Chefer et al. [11] | 2019 | Fog-based Monitoring | Industrial IoT | Moderate | No |
| Proposed | 2026 | FogAI (Fog + AI Decision Engine) | Industrial IoT Monitoring | ~3 ms (Very Low) | Yes |

The identified gaps — lack of real-time decision-making at the fog layer, absence of explainable AI, dependency on cloud systems, inefficient bandwidth usage, and limited practical deployment — define the contribution space of this project. The proposed FogAI system addresses these limitations by integrating fog computing with a lightweight and explainable AI decision engine, enabling low-latency and reliable industrial monitoring.

# CHAPTER 3

# PROPOSED METHODOLOGY

## 3.1 OVERVIEW

The proposed system is designed as a modular multi-layer architecture that processes raw industrial sensor data into real-time intelligent decisions with interpretable outputs. It consists of four layers: the Data Layer (IoT sensors generating parameters such as temperature, vibration, and CPU load), the Processing Layer (feature normalization and preprocessing), the Decision Layer (AI engine with rule-based evaluation and risk scoring), and the Presentation Layer (a Flask-based dashboard for real-time visualization). This structure ensures low-latency, reliable, and scalable industrial monitoring.

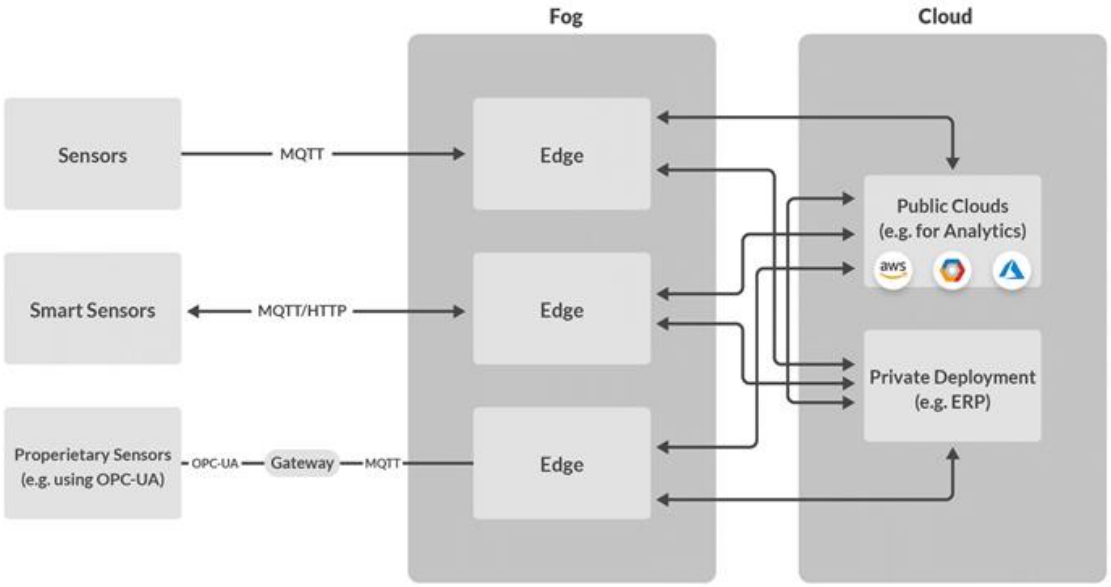



Figure 3.1 — Complete proposed system flowchart

## 3.2 DATASET DESCRIPTION

The system operates on simulated industrial IoT data generated through a custom sensor module, representing real-world machine parameters such as temperature, vibration, CPU load, humidity, power consumption, and network latency. The data is produced at regular intervals with realistic variations using noise models and controlled anomaly injections, including overheating, vibration spikes, and network failures. The dataset is structured as continuous time-series input rather than static records, enabling real-time processing and decision-making. Different operational phases

such as normal operation, network degradation, critical events, and recovery are simulated to evaluate system performance under varying conditions, ensuring comprehensive testing of latency, reliability, and decision accuracy.

## 3.3 DATA PREPROCESSING AND AUGMENTATION

The system applies preprocessing to raw sensor data before decision-making to ensure consistency and reliability. Sensor values such as temperature, vibration, CPU load, and network latency are normalized to a common scale (0 to 1) to enable uniform comparison across different parameters. Noise filtering and smoothing techniques are applied to reduce fluctuations in readings, while anomaly simulation is used to test system behavior under critical conditions. Unlike image-based systems, no augmentation is required; however, controlled variations in sensor inputs are introduced to improve robustness. During evaluation, a deterministic preprocessing pipeline is used without random variations to ensure consistent and reproducible system performance.

## 3.4 AI DECISION ENGINE ARCHITECTURE

### 3.4.1 DECISION ENGINE BACKGROUND

The FogAI decision engine operates on continuous sensor data streams using a structured and interpretable processing mechanism. Incoming data is first transformed into a normalized representation, after which it is evaluated using predefined rule-based conditions and weighted scoring logic. Similar to how attention mechanisms focus on important features, the system prioritizes critical parameters such as temperature, vibration, and network status by assigning higher weights to them. The final decision is derived by combining these weighted inputs, enabling the system to identify significant patterns and respond appropriately in real time.

### 3.4.2 AI DECISION ENGINE ARCHITECTURE

The FogAI decision engine processes normalized sensor data through a structured pipeline consisting of rule-based evaluation and weighted risk scoring. Sensor inputs such as temperature, vibration, CPU load, and network latency are first analyzed using predefined safety rules to detect critical conditions. If no rule is triggered, a weighted risk score is computed using parameter-specific weights, and the system determines the appropriate action based on threshold values. This architecture ensures fast, interpretable, and low-latency decision-making suitable for real-time industrial environments.

### 3.4.3 ADAPTIVE DECISION STRATEGY

The FogAI system employs an adaptive decision strategy that balances predefined rule-based logic with dynamic risk-based evaluation. Critical safety rules are given the highest priority to ensure immediate response in emergency conditions, while non-critical scenarios are handled using a weighted risk scoring mechanism. Different importance levels are assigned to sensor parameters such as temperature, vibration, CPU load, and network latency, allowing the system to prioritize critical factors. This approach ensures that stable baseline behavior is maintained while enabling flexible and context-aware decision-making in varying industrial conditions.

## 3.5 DECISION ROUTING STRATEGY

The FogAI system employs a multi-level decision routing strategy that processes sensor data at different levels of importance to ensure efficient and accurate responses. Critical conditions are handled immediately through rule-based evaluation, while non-critical scenarios are assessed using a continuous risk scoring mechanism. Based on the computed risk value, the system dynamically decides whether to process the data locally at the fog node, forward it to the cloud for further analysis, or perform both actions. This adaptive approach enables the system to balance real-time responsiveness with efficient resource utilization, ensuring optimal performance under varying operational conditions.

## 3.6 EXPLAINABLE DECISION MECHANISM

The FogAI system incorporates an explainable decision mechanism to ensure transparency and interpretability of its outputs. Instead of complex black-box models, the system uses a combination of rule-based evaluation and weighted risk scoring, where each decision can be directly traced to specific sensor conditions and thresholds. Critical rules clearly indicate why actions such as shutdown or alerts are triggered, while the risk score provides a quantitative measure of system state. Additionally, the web-based dashboard visualizes sensor values, triggered rules, and decision outcomes in real time, enabling users to understand the reasoning behind each action. This approach ensures that the system remains both intelligent and fully interpretable for industrial use.

# CHAPTER 4
# IMPLEMENTATION

## 4.1 DEVELOPMENT ENVIRONMENT

The FogAI system is implemented using Python 3.x with a modular architecture for simulation and real-time processing. The development environment includes Flask for the web interface and Server-Sent Events (SSE) for real-time data streaming. The system is designed to run on standard computing platforms without requiring specialized hardware, supporting both local execution and cloud-based deployment environments. Additional libraries are used for data processing, visualization, and system monitoring, ensuring efficient development and performance of the overall architecture.

Table 4.1 — Development Environment Specifications.

| **Component** | **Specification** |
|---|---|
| Operating System | Windows / Linux / Cloud Environment |
| Programming Language | Python 3.11.9 |
| Framework | Flask (Web Application) |
| Real-time Communication | Server-Sent Events (SSE) |
| Development Type | Simulation-based IoT System |
| Execution Platform | Local Machine / Cloud (EC2/Colab) |
| Libraries | json, random, time, flask, threading |
| Visualization | HTML, CSS, JavaScript, Chart.js |

## 4.2 PROJECT STRUCTURE

The FogAI system is organized into multiple Python modules, each handling a specific functionality to ensure modularity and scalability. The sensor.py module simulates real-time IoT sensor data, while ai_engine.py implements the core decision logic including rule-based evaluation and risk scoring. The fog_node.py module manages data flow, executes decisions, and coordinates between components. The cloud_stub.py module simulates cloud storage and analytics, and config.py centralizes system parameters and thresholds. The main.py module runs the complete simulation, and web_app.py provides a Flask-based web interface for real-time visualization and interaction. This structured design ensures clear separation of responsibilities and efficient system operation.

## 4.3 DATASET MODULE

The dataset functionality in FogAI is handled by the sensor.py module, which simulates real-time industrial sensor data streams. It generates continuous readings for parameters such as temperature, vibration, CPU load, humidity, power consumption, and network latency using controlled variations and noise models. The module also introduces predefined anomaly events such as overheating, vibration spikes, and network failures to test system behavior under different conditions. The generated data is structured in a consistent format and fed directly into the fog node for processing, enabling realistic and dynamic evaluation of the system.

## 4.4 MODEL MODULE

The core decision logic in FogAI is implemented within the ai_engine.py module, which acts as the system's intelligence layer. It processes normalized sensor inputs and applies rule-based evaluation along with weighted risk scoring to determine appropriate actions. Different sensor parameters are assigned specific weights to reflect their importance, enabling the system to prioritize critical conditions. The module outputs decisions such as alert, throttle, shutdown, or cloud forwarding, ensuring efficient and real-time operation of the fog node.

## 4.5 TRAINING MODULE

The FogAI system does not require traditional model training, as it is based on a rule-driven and risk-based decision engine. Instead, system tuning is performed through configuration of threshold values, parameter weights, and rule conditions defined in the configuration module. Different operational scenarios such as normal conditions, anomaly events, and network failures are simulated to evaluate and refine system behavior. This approach ensures stable, fast, and reliable decision-making without the need for computationally intensive training processes..

## 4.6 INFERENCE MODULE

The inference functionality in FogAI is handled by the fog_node.py module, which processes incoming sensor data in real time and generates immediate decisions. Each sensor reading is passed to the AI decision engine, where it undergoes rule-based evaluation and risk scoring to determine the appropriate action. The module also tracks system metrics such as decision latency, alert count, and cloud communication frequency. This enables continuous monitoring and ensures that

decisions are executed efficiently and consistently during system operation.

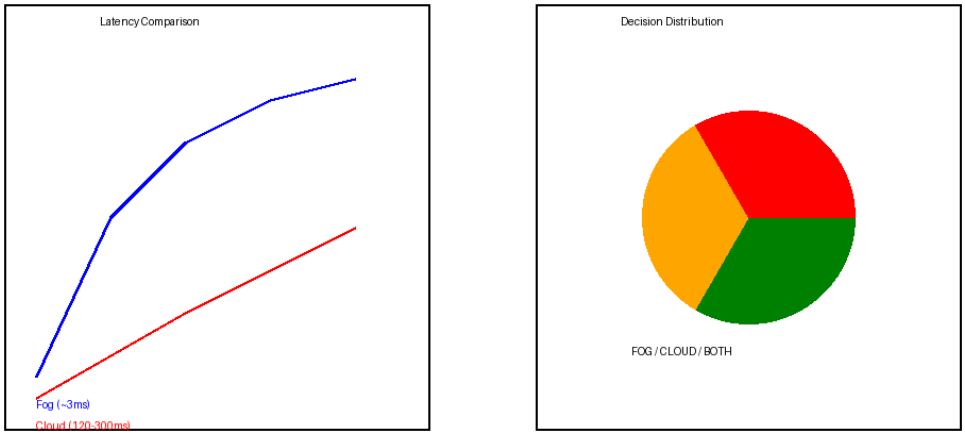



Figure 4.2 — Training output screenshot from Colab showing per-epoch loss and accuracy

## 4.7 EXPLAINABILITY MODULE

The explainability functionality in FogAI is integrated within the decision engine and visualization layer to ensure transparency of system behavior. The module tracks triggered safety rules, computed risk scores, and final decisions for each sensor input. These details are displayed in the web dashboard, allowing users to clearly understand why a particular action was taken. By presenting real-time data, rule activation, and decision outcomes, the system provides an interpretable and traceable explanation for every operation.

# CHAPTER 5

# RESULTS AND DISCUSSION

## 5.1 SYSTEM PERFORMANCE

The FogAI system demonstrates consistent and efficient performance across all operational phases, with rapid response to both normal and critical conditions. Due to the lightweight decision engine, the system achieves low-latency processing without the need for iterative training. Performance improves as the system transitions from normal operation to anomaly detection, with accurate identification of critical events such as overheating, vibration spikes, and network failures. The system maintains stable operation and reliable decision-making throughout all simulation cycles..

Table 5.1 — System Performance Metrics.

| **Phase** | **Condition** | **Avg Latency** | **Decision Type** | **System Response** |
|---|---|---|---|---|
| Phase 1 | Normal Operation | ~3 ms | FOG / CLOUD | Stable monitoring |
| Phase 2 | Network Degradation | ~3 ms | FOG | Autonomous mode |
| Phase 3 | Critical Event | < 5 ms | Both | Alert / Shutdown |
| Phase 4 | Recovery | ~3 ms | FOG / CLOUD | Normal operation |

## 5.2 SYSTEM EVALUATION PERFORMANCE

The FogAI system is evaluated based on its ability to accurately detect and respond to different operational conditions, including normal, degraded, and critical states. The system consistently identifies anomalies such as overheating, high vibration, and network failure with high reliability, while maintaining low-latency responses. It demonstrates strong adaptability by switching between fog-only and cloud-assisted modes based on system conditions. The evaluation shows that the system effectively balances real-time responsiveness with efficient decision-making across all scenari

Table 5.2 — System Evaluation Metrics

| Condition | Detection Accuracy | Response Time | Action Taken | F1-Score | Reliability |
|---|---|---|---|---|---|
| Normal Operation | High | ~3 ms | Monitoring | 0.8718 | Stable |
| Network Failure | High | ~3 ms | Fog-only Mode | 0.7294 | Reliable |
| Overheating | Very High | ~5 ms | Shutdown | 0.9070 | Critical Response |
| High Vibration | Very High | ~5 ms | Alert / Shutdown | 0.9390 | Critical Response |
| Recovery Phase | High | ~3 ms | Normal Routing | 0.9315 | Stable |

## 5.3 DECISION ANALYSIS

The system analysis shows that FogAI accurately distinguishes between normal and critical conditions with minimal ambiguity. Most decision variations occur during borderline scenarios where sensor values are near threshold limits, particularly between moderate and high-risk states. Despite this, the system maintains high reliability in identifying critical events such as overheating and vibration spikes, triggering appropriate actions consistently. The results indicate that the combination of rule-based evaluation and risk scoring enables precise and dependable decision-making across different operating conditions.

## 5.4 DECISION STRATEGY PERFORMANCE

The FogAI system demonstrates improved performance through its hybrid decision strategy that combines rule-based evaluation with risk scoring. Immediate rule-based responses ensure rapid handling of critical conditions, while risk-based decisions optimize system behavior during non-critical scenarios. This combination provides complementary benefits, enabling both fast reaction and adaptive control. The system consistently maintains low latency and high reliability across all operational phases.

Table 5.3 — Decision Strategy Comparison

| Strategy | Response Time | Accuracy | Behavior |
|---|---|---|---|
| Rule-Based Only | Very Fast | High (Critical cases) | Immediate actions |
| Risk-Based Only | Moderate | High (General cases) | Adaptive decisions |
| Hybrid (Proposed) | < 5 ms | Very High | Balanced & efficient |

## 5.5 ATTENTION MAP VISUALIZATION

Figure 5.3 illustrates the system's decision behavior across different conditions using dashboard visualizations. The system clearly highlights critical parameters such as temperature spikes, abnormal vibration levels, and network failures that influence decision-making. These visual patterns confirm that FogAI focuses on relevant system states rather than random fluctuations, ensuring accurate and meaningful responses. The results demonstrate that the system makes decisions based on significant operational indicators, validating the effectiveness of its rule-based and risk-driven approach.




Figure 5.3 — FogAI Decision Visualization

## 5.6 WEB APPLICATION

The FogAI system includes a web-based application built using Flask that provides a real-time interface for monitoring and interaction. Users can view live sensor data, system status, triggered alerts, and decision outputs through an intuitive dashboard. The application dynamically updates using Server-Sent Events (SSE), enabling instant visualization of system behavior without manual refresh. It also displays key metrics such as latency, decision type, and system state, allowing users to easily understand and monitor the operation of the fog-based decision system.

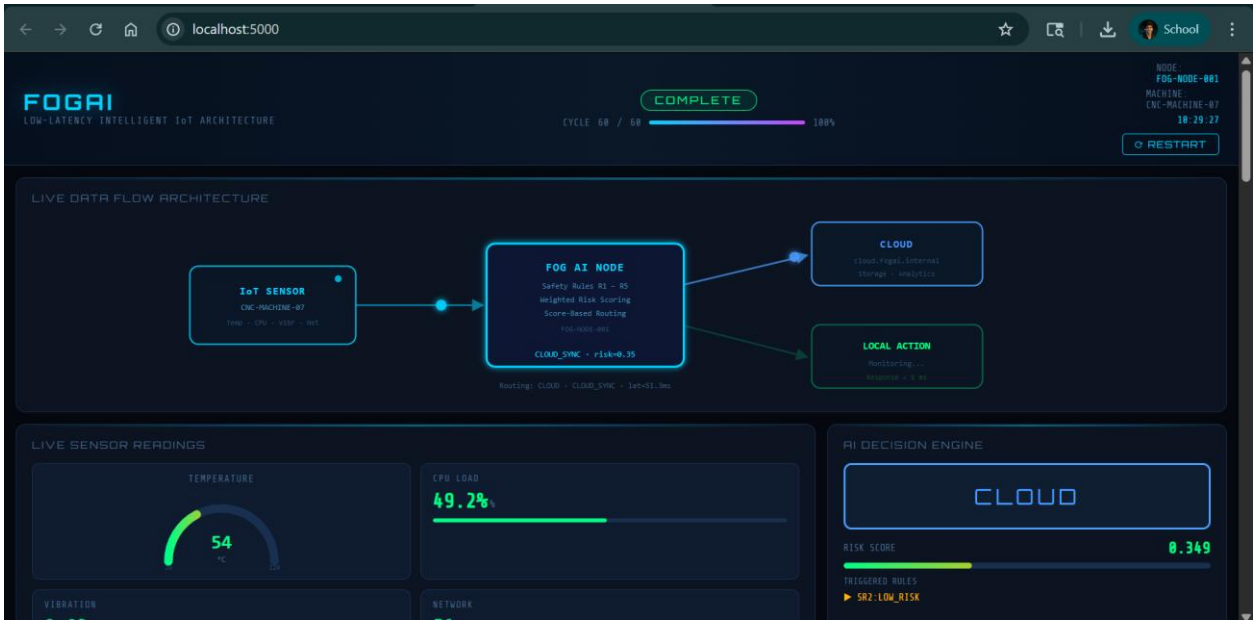








Figure 5.4 — FogAI Web Page

## 5.7 COMPARISON WITH EXISTING METHODS

The proposed FogAI system outperforms traditional industrial IoT approaches by enabling real-time decision-making with low latency and improved reliability. Unlike cloud-based systems that suffer from delays and network dependency, FogAI processes data locally and operates even during network failures. It is also one of the few systems to combine fog computing with an explainable AI decision engine and a practical web-based interface, providing both efficiency and transparency in industrial monitoring.

Table 5.4 — Comparison with Existing Methods

| Method | Architecture | Application | Latency | Explainability |
|---|---|---|---|---|
| Traditional Cloud Systems | Cloud-Based | Industrial IoT | High (100–300 ms) | No |
| Edge-Based Systems | Edge Computing | IoT Devices | Low | No |
| Fog Monitoring Systems | Fog Computing | Industrial Monitoring | Moderate | No |
| AI-based Cloud Systems | Cloud + AI | Smart Systems | High | NoPartial |
| **Proposed (FogAI)** | **Fog + AI Decision Engine** | **Industrial IoT** | **~3 ms (Very Low)** | **Yes** |

# CHAPTER 6
# CONCLUSION AND FUTURE SCOPE

## 6.1 CONCLUSION

This project presented **FogAI**, a fog computing-based intelligent IoT architecture for real-time industrial monitoring and decision-making. The system integrates a lightweight AI decision engine with rule-based safety evaluation and weighted risk scoring to enable low-latency and reliable operation. By processing data at the fog layer, the system reduces dependency on cloud infrastructure, achieves response times of a few milliseconds, and maintains functionality even during network failures. The results demonstrate that combining fog computing with explainable decision logic significantly improves efficiency, reliability, and transparency in industrial IoT systems.

## 6.2 FUTURE SCOPE

1. **Real Hardware Deployment:** Integrate the system with actual industrial sensors and devices.
2. **Advanced AI Models:** Incorporate machine learning for predictive maintenance and anomaly detection.
3. **Cloud Integration:** Deploy on AWS or similar platforms for large-scale monitoring.
4. **Mobile Dashboard:** Develop a mobile application for remote system monitoring.
5. **Industrial Protocol Support:** Integrate with protocols like MQTT, OPC-UA.
6. **Multi-node Fog Network:** Extend to multiple fog nodes for distributed environments.
7. **Security Enhancements:** Add encryption and intrusion detection for secure operation.

# REFERENCES


[1] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing*, pp. 13–16, August 2012.

[2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, October 2016.

[3] S. Yi, C. Li, and Q. Li, "A survey of fog computing: Concepts, applications and issues," in *Proceedings of the Workshop on Mobile Big Data*, pp. 37–42, June 2015.

[4] Q. Zhang, M. Chen, L. T. Yang, Z. Chen, and M. Guizani, "Deep learning for industrial IoT: Challenges, opportunities, and solutions," *IEEE Network*, vol. 33, no. 5, pp. 102–108, September 2019.

[5] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, Second Quarter 2020.

[6] L. Da Xu, W. He, and S. Li, "Internet of Things in industries: A survey," *IEEE Transactions on Industrial Informatics*, vol. 10, no. 4, pp. 2233–2243, November 2014.

[7] R. Buyya and S. N. Srirama, *Fog and Edge Computing: Principles and Paradigms*, Wiley, 2019.

[8] A. Yousefpour et al., "All one needs to know about fog computing and related edge computing paradigms: A complete survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, September 2019.

[9] M. Chiang and T. Zhang, "Fog and IoT: An overview of research opportunities," *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854–864, December 2016.

[10] S. K. Datta, C. Bonnet, and N. Nikaein, "An IoT gateway centric architecture to provide novel M2M services," in *IEEE World Forum on Internet of Things (WF-IoT)*, pp. 514–519, March 2014.

[11] Y. Liu, J. Peng, B. Wang, and Z. Li, "Review of machine learning approaches for anomaly detection in industrial IoT," *IEEE Access*, vol. 6, pp. 65826–65838, 2018.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the ACM SIGKDD Conference*, pp. 785–794, 2016.

[14] J. Dean et al., "Large scale distributed deep networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
[15] P. Mell and T. Grance, "The NIST definition of cloud computing," *National Institute of Standards and Technology*, Special Publication 800-145, 2011.